10 INHERITANCE • THEORY

04 ambiguity diamond problem and virtual base

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

04 — Ambiguity in Multiple Inheritance, the Diamond Problem, and `virtual` Base Classes

1. Member Name Ambiguity in Multiple Inheritance

Suppose two independent base classes `B1` and `B2` both have a function named `show()`, and class `D` inherits from both:

CPP

```cpp
class B1 {
public:
    void show() { cout << "From B1" << endl; }
};

class B2 {
public:
    void show() { cout << "From B2" << endl; }
};

class D : public B1, public B2 {
};
```

If `main()` writes `D obj; obj.show();`, the compiler reports an **ambiguity error**: `request for member 'show' is ambiguous`.

- **Resolution**: Qualify the call using the scope resolution operator (`::`):

  CPP

  ```cpp
  obj.B1::show();
  obj.B2::show();
  ```

2. The Diamond Problem (Multipath Inheritance Duplication)

Consider a 4-class diamond hierarchy:

- Top base class: `person` (contains `int age;`)
- Two intermediate classes: `student : public person` and `employee : public person`
- Bottom class: `teaching_assistant : public student, public employee`

What Goes Wrong Without `virtual` Inheritance?

An object of `teaching_assistant` contains **TWO separate** `person` **subobjects** in memory:

1. One `person::age` inherited via the `student` path (`ta.student::age`).
2. A second, duplicate `person::age` inherited via the `employee` path (`ta.employee::age`).

Writing `ta.age = 22;` fails to compile with `error: request for member 'age' is ambiguous` !
Moreover, the `person` constructor runs **twice**.

---

3. Solution: `virtual` Base Classes (`virtual public`)

To ensure that `teaching_assistant` shares **only ONE single common** `person` **subobject**, the intermediate classes (`student` and `employee`) must inherit from `person` using the `virtual` keyword:

CPP

```cpp
class person {
public:
    int age;
};

class student : virtual public person { };
class employee : virtual public person { };

class teaching_assistant : public student, public employee { };
```

Rules of `virtual` Base Classes (`[class.mi]`)

1. `virtual public` (or `public virtual` — both orders are identical) guarantees that only **one shared instance** of the `person` base subobject exists inside `teaching_assistant` .
2. `ta.age = 22;` is now completely **unambiguous** and valid!
3. **Constructor Rule for Virtual Base Classes**: Virtual base class constructors are always invoked **before** any non-virtual base class constructors, and the **most-derived class** (`teaching_assistant`) is responsible for initializing the virtual base class (`person`).